Performance and Accuracy of AI Weather Forecasting Models

Pangu-Weather and FourCastNet on Indian Flood Events,
and LSTM Streamflow Prediction with NeuralHydrology

Arijit (240175)
IIT Kanpur

October 10, 2026

Abstract

This report tests two published AI weather forecasting models, Pangu-Weather (Huawei, *Nature*, 2023) and FourCastNet (NVIDIA/LBNL, 2022), on five real flood events in India: Mumbai (2005 and 2024), Delhi (2023), Chennai (2015) and Idukki, Kerala (2018). Each model was started from real ERA5 weather data about two days before the flood, so that it had to forecast the event.

FourCastNet forecasts rainfall, so its predictions were compared with the observed rainfall over the same time window as each observation. It underestimated every flood: by 49% for the three-day Idukki event and by 85–98% for the other four. The size of the error did not depend on how large the flood was ($r = -0.19$, $p = 0.76$) or on how many days in advance the forecast was started. To find where this error comes from, the predictions were also compared with two other rainfall datasets for the same grid cell: IMD's gauge-based gridded rainfall and ERA5, the data FourCastNet was trained on. FourCastNet stayed within 4–20% of ERA5 in three of five cases, but ERA5 itself was 31–85% below the IMD grid. Most of the error is therefore inherited from the training data, which cannot represent intense local monsoon rain.

Pangu-Weather does not forecast rainfall at all, so its pressure and humidity forecasts were tested as early-warning signs. It produced realistic weather patterns, but neither signal matched how severe the floods were (pressure: $r = -0.45$, $p = 0.45$; humidity: $r = -0.61$, $p = 0.27$; $n = 5$).

In a second part, an LSTM model was trained with the NeuralHydrology framework to predict daily river flow. After reproducing the CAMELS US benchmark (test NSE 0.776), it was applied to an Indian catchment (CAMELS-IND basin 3002) with three different weather inputs: NCEP FNL global analyses from NSF NCAR GDEX (dataset `d083002`), FNL plus observed rainfall, and the catchment's own CAMELS-IND data. FNL has no rainfall before June 2019, so atmospheric and soil moisture were used instead. CAMELS-IND data gave the best predictions (KGE 0.38); FNL alone caught the timing of the monsoon response (correlation 0.63) but overestimated flow (KGE 0.01). The work also found wrong values in the basin's streamflow record that had hidden all differences between the models.

Contents

1 Introduction **2**
1.1 Pangu-Weather 3

2 Dataset **3**
2.1 ERA5 3
2.2 Two Roles of ERA5 3
2.3 Data Access 4
2.4 Rainfall Data Used for Checking the Forecasts 4
2.5 Picking Values for a City 4

3 Methodology **4**
3.1 Tools . . . 4
3.2 Flood Events . . . 5
3.3 What Each Model Can Be Tested On . . . 5
3.4 Reading FourCastNet's Rainfall . . . 5
3.5 Running Pangu-Weather on the T4 GPU . . . 6
3.6 Pangu-Weather Warning Signs . . . 6

4 Results **6**
4.1 FourCastNet: Predicted vs Observed Rainfall . . . 6
4.2 FourCastNet: Does Error Grow with Flood Size? . . . 6
4.3 FourCastNet: Does Starting Earlier Make It Worse? . . . 7
4.4 FourCastNet: When Did It Expect Rain? . . . 8
4.5 FourCastNet: Timing Check for Mumbai 2024 . . . 8
4.6 FourCastNet: Where Does the Error Come From? . . . 9
4.7 Pangu-Weather: Pressure and Humidity Over Time . . . 10
4.8 Pangu-Weather: Warning Signs vs Flood Size . . . 11
4.9 Computational Performance . . . 12

5 Discussion **12**

6 Limitations **12**

7 Streamflow Prediction with NeuralHydrology **13**
7.1 Objective . . . 13
7.2 Framework and Model . . . 13
7.3 Evaluation Metrics . . . 14
7.4 Stage 1: Checking the Pipeline on CAMELS US . . . 14
7.5 Stage 2: CAMELS-IND with Its Own Weather Data . . . 14
7.6 Stage 3: NCEP FNL Data from GDEX (d083002) . . . 15
7.6.1 Dataset . . . 15
7.6.2 Data Access and Processing . . . 15
7.6.3 Data Access and Processing . . . 15
7.6.4 Comparison with CAMELS-IND Data . . . 16
7.6.5 Experimental Design . . . 16
7.6.6 Results . . . 17
7.6.7 Discussion . . . 18
7.6.8 Limitations . . . 19
7.6.9 Conclusion . . . 19

8 Conclusion **20**

1 Introduction

Normal weather forecasts come from numerical weather prediction (NWP): large physics simulations of the atmosphere run on supercomputers. In the last few years, a new approach has appeared. Deep learning models are trained on decades of past weather data and learn how the weather changes from one moment to the next. They can match the accuracy of physics models for many variables while running thousands of times faster.

This report looks at two such models:

- **Pangu-Weather** (Bi et al., 2023): a 3D transformer that treats the atmosphere as a 3D block of data. It forecasts temperature, humidity, wind, air pressure and geopotential height. It does *not* forecast rainfall.
- **FourCastNet** (Pathak et al., 2022): a transformer based on Fourier transforms, with an extra module that forecasts rainfall.

Both models were trained by their authors on decades of ERA5 data (Section 2). In this work they were only *run* (inference) using the published trained weights; they were not retrained.

Section 7 then moves from the weather to its effect on rivers: an LSTM model that predicts daily river flow from weather inputs, trained with the NeuralHydrology framework.

1.1 Pangu-Weather

Pangu-Weather looks at all heights of the atmosphere together, as one 3D block, instead of treating each height separately. This lets it learn how the layers affect each other, for example moist air near the ground under a pressure dip higher up. It also knows where each grid cell is on the globe, because cells near the equator and near the poles behave differently. The model has about 256 million parameters and was trained on 39 years of hourly ERA5 data (1979–2017).

Inputs and outputs. At each step, Pangu-Weather takes the full state of the atmosphere at one moment and predicts the same fields for a later time:

- **Upper air (5 variables at 13 heights):** geopotential (z), specific humidity (q), temperature (t) and wind (u, v) at 1000, 925, 850, 700, 600, 500, 400, 300, 250, 200, 150, 100 and 50 hPa.
- **Surface (4 variables):** sea-level pressure (`msl`), wind at 10 m (`u10`, `v10`) and temperature at 2 m (`t2m`).

That is $5 \times 13 + 4 = 69$ global maps on a 0.25° (about 25 km) grid. **Rainfall is neither an input nor an output**, which decides how Pangu-Weather could be tested here (Section 3.3).

Fewer, bigger steps. Every prediction made from a previous prediction adds a little error, and these errors pile up. Pangu-Weather therefore has separate models for different step sizes and uses the biggest steps possible. The version used here has a 24-hour and a 6-hour model. For a 60-hour forecast, hours 24 and 48 come from the 24-hour model (jumping straight from hour 0 and hour 24), and the hours in between come from the 6-hour model. Hour 48 is then only two stacked predictions instead of eight.

2 Dataset

2.1 ERA5

All model inputs come from **ERA5**, the global weather reanalysis made by ECMWF. ERA5 combines real observations (stations, satellites, weather balloons, ships, aircraft) with a physics model to give a complete, hourly weather map of the whole Earth from 1940 to about five days ago, on a 0.25° (about 25 km) grid, the same grid as both models. Upper-air fields come from the *pressure levels* product and surface fields from the *single levels* product.

2.2 Two Roles of ERA5

ERA5 is used in two different ways:

1. **Training data (done by the model authors, not here):** both models learned how weather changes from about 40 years of ERA5.

2. **Starting point (done here):** for each forecast, one ERA5 snapshot of the whole globe (69 fields at one date and time) was given to the trained model. The model then predicted forward step by step, using each prediction as the input for the next, until it reached the required time.

Even for floods inside the training period (such as Mumbai 2005), the model has not memorised the event. It was trained on millions of general "weather now → weather later" examples, not on specific dates.

2.3 Data Access

ERA5 was downloaded through the **Copernicus Climate Data Store (CDS) API** with a personal API key. The `ai-models` toolkit sends the request, waits for CDS to prepare the data and downloads it in GRIB format. Each starting snapshot was about 130 MB.

2.4 Rainfall Data Used for Checking the Forecasts

Three rainfall sources were used to check FourCastNet's predictions:

- **Reported rainfall:** values from IMD rain gauges or published event reports (Table 1). These are single points, a city average or a river-basin total.
- **IMD gridded rainfall** (Pai et al., 2014): daily rainfall built from about 7,000 IMD rain gauges on the same 0.25° grid as the models. This is measured rain on the same grid size, so it is the fairest comparison. IMD's rainfall day runs from 08:30 to 08:30 IST and is labelled with the date of the morning it is measured. The files were downloaded from IMD Pune's website and read with the `imdlib` Python package.
- **ERA5 rainfall** for the same grid cell: the rainfall FourCastNet was trained to reproduce.

2.5 Picking Values for a City

The models and ERA5 give values on a grid, not at points. For each city the nearest grid cell (about 25 km across) to its coordinates was used. A rain gauge measures one spot, while a grid cell is an average over a large area, so a gauge can record much more rain during an intense local storm. Section 4.6 measures how big this effect is.

3 Methodology

3.1 Tools

The original authors share their trained weights through Google Drive, Baidu Netdisk and Globus, which are hard to use in an automated pipeline. Instead, ECMWF's `ai-models` toolkit with the `ai-models-panguweather` and `ai-models-fourcastnet` plugins was used. It downloads the correct weights from ECMWF's servers, fetches the starting data from CDS, runs the model and writes standard GRIB output. Everything ran on a Google Colab GPU (NVIDIA T4, about 15 GB of GPU memory).

Some fixes were needed to make the old, no-longer-maintained toolkit work with current libraries: pinning `earthkit-data` below version 1.0, installing `onnxruntime-gpu` so that Pangu-Weather actually used the GPU, and allowing PyTorch to load the FourCastNet checkpoint (`weights_only=False`, safe here because the file comes from ECMWF's own server).

3.2 Flood Events

Five real, well-documented floods in four cities were chosen, covering different sizes and regions (Table 1). Each forecast started about two days before the flood, so the models had to predict it without seeing any data from the flood day. Both models were run on the same events.

Table 1: Flood events used in this study

Case	City	Start date	Observed rainfall	Notes
Mumbai_2005	Mumbai	24 Jul 2005	944 mm / 24h	IMD Santacruz; Mumbai's wettest day on record
Mumbai_2024	Mumbai	6 Jul 2024	300 mm / 6h	IMD red alert, 01:00–07:00 IST
Delhi_2023	Delhi	7 Jul 2023	153 mm / 24h	IMD Safdarjung; most July rain in one day since 1982
Chennai_2015	Chennai	29 Nov 2015	286 mm / 24h	City average (77–494 mm across stations)
Idukki_2018	Idukki, Kerala	13 Aug 2018	588 mm / 3 days	Periyar river basin; peak of the 2018 Kerala floods

All forecasts started at 00:00 UTC. FourCastNet's forecast length was chosen so that it covers the whole observation window: 60 hours for Mumbai 2024 and Delhi, 78 hours for Mumbai 2005 and Chennai, and 102 hours for Idukki. Pangu-Weather used 60 hours (96 for Idukki).

3.3 What Each Model Can Be Tested On

Pangu-Weather does not forecast rainfall at all (Section 1.1). **FourCastNet** has a separate rainfall module (`precip.ckpt`) and does forecast rainfall (`tp`). The two models were therefore tested in different ways:

- **FourCastNet:** its predicted rainfall was compared directly with observed rainfall (Sections 4.1–4.6).
- **Pangu-Weather:** since it cannot predict rain, two common warning signs of heavy rain were used instead, falling air pressure and rising humidity at about 1.5 km (850 hPa). These were checked for being stronger before bigger floods (Section 4.7). This tests early warning, not rainfall accuracy.

3.4 Reading FourCastNet's Rainfall

Two details matter when reading FourCastNet's rainfall from the GRIB files:

1. **Loading the field.** The `cfgrib` library silently drops fields that do not fit the height setup of the others, including rainfall and 2 m temperature. These were loaded separately by their ID numbers (228 and 167).
2. **Running totals.** FourCastNet stores rainfall as a *running total since the forecast started*: at every 6-hour step it writes all the rain so far. So the total rain between two times is the running total at the end minus the running total at the start, and the rain in one 6-hour step is the difference between two neighbouring values. (Adding up all the stored values would count the same rain many times.) When an observation window starts or ends inside a 6-hour step, the running total was linearly interpolated. A check confirmed that the running total never decreased.

Same time window. Each prediction was measured over exactly the same window as its observation, converted to UTC (IST is UTC+5:30). IMD's rainfall day (08:30 to 08:30 IST) is 03:00 to 03:00 UTC. The windows were: Mumbai 2005, 26 Jul 03:00 – 27 Jul 03:00; Mumbai 2024, 7 Jul 19:30 – 8 Jul 01:30; Delhi, 8 Jul 03:00 – 9 Jul 03:00; Chennai, 1 Dec 03:00 – 2 Dec 03:00; Idukki, 14 Aug 03:00 – 17 Aug 03:00 (three IMD days, an assumption based on the reported 15–17 August).

3.5 Running Pangu-Weather on the T4 GPU

By default, the `ai-models` plugin loads both the 24-hour and 6-hour Pangu models onto the GPU together. This needs more memory than the T4 has, so the run crashed at the first step (it failed to get about 1.85 GB more). The plugin was changed so that the 6-hour model stays on the GPU (with a setting, `kSameAsRequested`, that stops it reserving extra memory) and the 24-hour model runs on the CPU. The 24-hour model is used only every fourth step, so most of the work still runs on the GPU. This only changes where the computation happens, not the model or its predictions.

3.6 Pangu-Weather Warning Signs

For each city, two values were taken from the nearest grid cell at every 6-hour step: sea-level air pressure (hPa), where a fall suggests a rain-bearing low-pressure system, and humidity at about 1.5 km (850 hPa, g/kg), where a rise means more moisture available for rain. In the tropics, air pressure naturally rises and falls twice a day, so the average of the last 24 hours was compared with the average of the first 24 hours; each 24-hour average covers one full daily cycle, so the cycle cancels out. A useful warning sign would show falling pressure and rising humidity, both stronger before bigger floods. This was tested with the Pearson correlation.

4 Results

4.1 FourCastNet: Predicted vs Observed Rainfall

Table 2 compares FourCastNet's predicted rainfall with the observed rainfall over the same time window.

Table 2: FourCastNet predicted vs observed rainfall, same time window

Case	Observed (mm)	Predicted (mm)	Error (mm)	Error (%)	Whole forecast (mm)
Delhi_2023	153	10.8	−142.2	−92.9	17.0
Chennai_2015	286	44.3	−241.7	−84.5	120.5
Mumbai_2024	300	6.3	−293.7	−97.9	57.4
Idukki_2018	588	302.8	−285.2	−48.5	373.9
Mumbai_2005	944	59.8	−884.2	−93.7	174.5

FourCastNet predicted too little rain for **every** flood. The best result was the three-day Idukki event (−49%), a long, widespread monsoon spell. The worst were the short, intense events: Mumbai 2024's six-hour burst (−98%), Mumbai 2005 (−94%) and Delhi (−93%). The last column shows the total rain over the whole forecast for reference; even that is far below the observed amounts.

4.2 FourCastNet: Does Error Grow with Flood Size?

The Pearson correlation between flood size (observed rainfall) and absolute error was:

$r = -0.192$, $p = 0.757$ $(n = 5)$

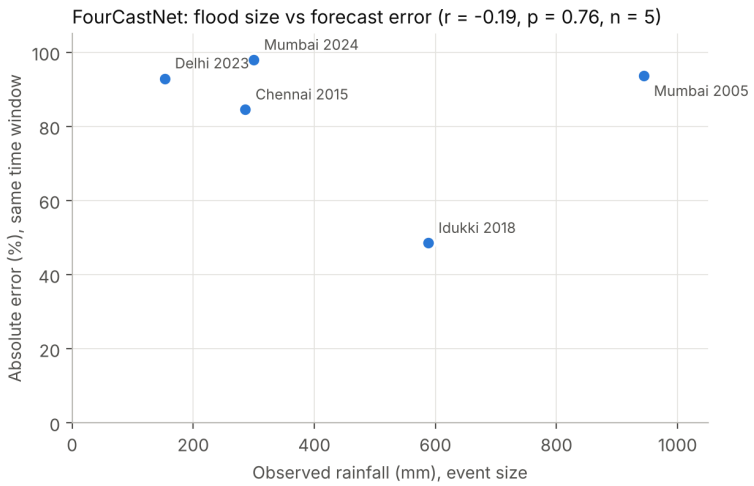


Figure 1: FourCastNet: observed rainfall (flood size) vs absolute error over the same time window.

There is no relationship. The model does not get worse as floods get bigger; it misses *all* of them by a large amount (85–98%, apart from Idukki). With only five events this is an indication, not a proof.

4.3 FourCastNet: Does Starting Earlier Make It Worse?

To test the effect of forecast lead time, the Mumbai 2005 forecast was started 1, 2, 3 and 5 days before the flood. Rainfall was always measured over the same flood-day window (26 Jul 03:00 – 27 Jul 03:00 UTC), so only the start date changes (Table 3).

Table 3: Mumbai 2005: flood-day rainfall vs forecast start date (observed: 944 mm)

Started (days before)	Hours ahead	Predicted (mm)	Error (%)
1	51	40.8	−95.7
2	75	59.8	−93.7
3	99	66.8	−92.9
5	147	53.6	−94.3

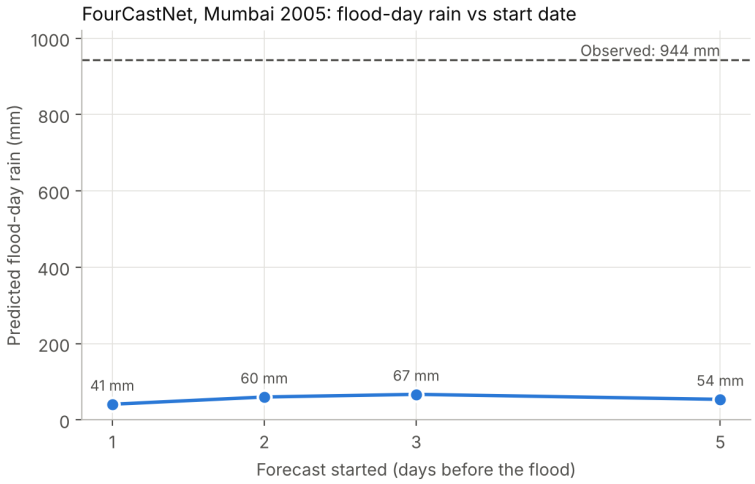


Figure 2: FourCastNet, Mumbai 2005: predicted flood-day rainfall vs how many days before the flood the forecast started.

The prediction stays between 41 and 67 mm whether the forecast starts one day or five days before the flood; the one-day start is even the lowest. So the error does not come from forecasting too far ahead. The model simply produces gentle, spread-out rain instead of extreme rain, whatever the start date.

4.4 FourCastNet: When Did It Expect Rain?

Each forecast was split into its 6-hour steps to see *when* the model expected rain. Table 4 shows Mumbai 2024; all five cases are in the accompanying notebook.

Table 4: FourCastNet 6-hourly rainfall, Mumbai 2024

Forecast hour	Time (UTC)	Rain in this 6 h (mm)	Running total (mm)
6	06 Jul 06:00	5.2	5.2
12	06 Jul 12:00	5.2	10.4
18	06 Jul 18:00	4.4	14.8
24	07 Jul 00:00	4.5	19.3
30	07 Jul 06:00	5.2	24.5
36	07 Jul 12:00	5.1	29.6
42	07 Jul 18:00	5.4	35.0
48	08 Jul 00:00	5.7	40.7
54	08 Jul 06:00	8.1	48.8
60	08 Jul 12:00	8.7	57.4

The model predicted steady, light rain of about 5–9 mm every 6 hours, slowly increasing towards the flood day, but with no burst. The same pattern appears in every case. The largest 6-hour amount predicted in any case was about 32 mm (Idukki), and for Mumbai 2005 about 18 mm, while the observed 944 mm in 24 hours means about 236 mm per 6 hours on average.

4.5 FourCastNet: Timing Check for Mumbai 2024

Mumbai 2024 is the only case where the exact timing of the heavy rain is known: 300 mm between 01:00 and 07:00 IST on 8 July (19:30 UTC on 7 July to 01:30 UTC on 8 July). Over

this window FourCastNet predicted only **6.3 mm**. Its wettest 6 hours were 06:00–12:00 UTC on 8 July (11:30–17:30 IST), with 8.7 mm: about 10 hours later than the real burst and far weaker. That slice is also the last step of the forecast, so the model's peak may come even later. The model sensed that rain was coming, but missed both the timing and the strength of the burst.

4.6 FourCastNet: Where Does the Error Come From?

A gauge measures one point, the IMD grid and ERA5 are 25 km averages, and FourCastNet learned from ERA5. Comparing all four for the same window shows where the rain is lost (Table 5, Figure 3).

Table 5: Rainfall from four sources over the same window (mm), and the gaps between them

Case	Gauge	IMD grid	ERA5	FourCastNet	IMD grid vs gauge	ERA5 vs IMD grid	FourCastNet vs ERA5
Mumbai_2005	944	423.4	74.3	59.8	−55%	−82%	−20%
Delhi_2023	153	61.4	34.9	10.8	−60%	−43%	−69%
Chennai_2015	286	329.3	50.6	44.3	+15%	−85%	−12%
Idukki_2018	588	455.5	314.9	302.8	−23%	−31%	−4%
Mumbai_2024	300	–	2.5	6.3	–	–	+3.8 mm

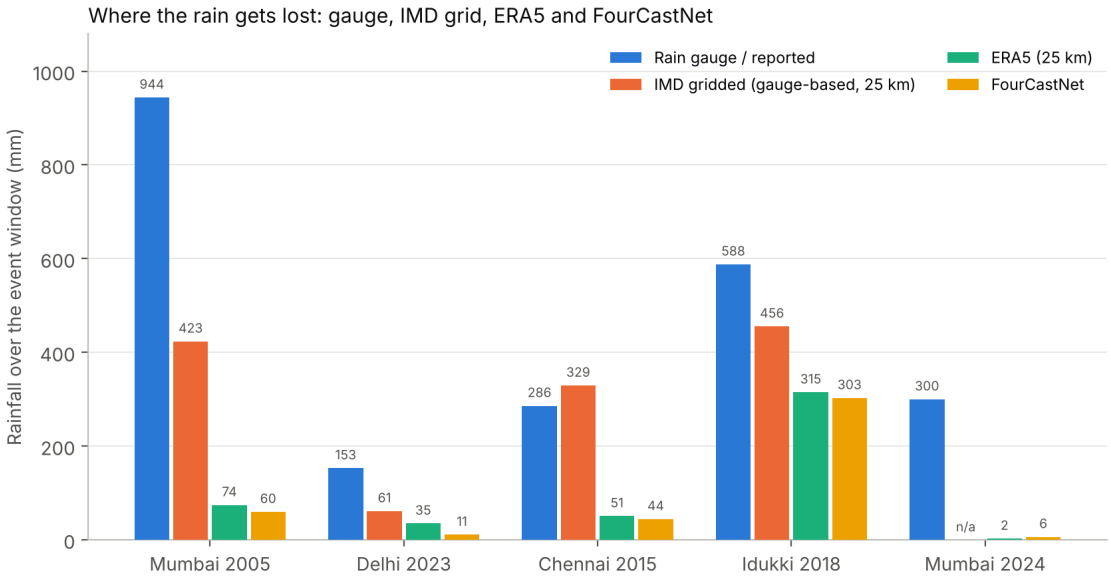


Figure 3: Rainfall over each event window from the gauge or report, the IMD gauge-based grid, ERA5 and FourCastNet. The IMD grid is daily, so it cannot be compared for the six-hour Mumbai 2024 window.

Reading the table from left to right:

1. **Gauge → IMD grid (point vs 25 km area):** averaging over a grid cell loses part of the rain (−23% to −60%). Chennai is +15% because its reported value was already a city average.

2. **IMD grid → ERA5 (the biggest loss):** even at the same grid size, ERA5 has 31–85% less rain than the measured IMD grid. ERA5's rainfall is computed by a weather model, and it cannot reproduce intense monsoon rain.

3. **ERA5 → FourCastNet (the model's own error):** FourCastNet stays within 4–20% of ERA5 in three cases. Delhi (−69%) is the one case where the model itself did clearly worse. For Mumbai 2024, ERA5 had only 2.5 mm in the burst window, so neither ERA5 nor FourCastNet saw the burst; the difference is under 4 mm.

For Mumbai 2005, for example: 944 mm at the gauge, 423 mm in the IMD grid cell, 74 mm in ERA5 and 60 mm from FourCastNet. Most of the rain is lost between the IMD grid and ERA5, before the AI model is involved.

The fairest accuracy figure is FourCastNet against the IMD grid (measured rain, same grid size): −86% (Mumbai 2005), −82% (Delhi), −87% (Chennai) and −34% (Idukki).

4.7 Pangu-Weather: Pressure and Humidity Over Time

Figure 4 shows Pangu-Weather's forecast of sea-level pressure and 850 hPa humidity at each city. Two features show that the output is physically realistic:

- **Correct pressure levels for each region.** Mumbai and Delhi (July) sit at about 999–1006 hPa, as expected under the July monsoon trough over north and central India. Chennai (a November, north-east monsoon event) and Idukki (southern peninsula) sit at about 1010–1015 hPa.
- **Twice-daily pressure cycle.** Every case shows a regular rise and fall of about 2–3 hPa, with highs at 06 and 18 UTC (about 11:30 and 23:30 IST), matching the known tropical pressure tide.

The warning signs differed a lot between cases:

- **Mumbai_2005:** the clearest pressure fall (daily lows from about 1004 to 1001 hPa), but humidity rose only in the first 12 hours and then stayed flat at about 14.4–14.7 g/kg.
- **Mumbai_2024:** pressure rose slightly, but humidity climbed steadily from about 14.6 to 15.3 g/kg between hours 24 and 60, the clearest moisture build-up of any case.
- **Delhi_2023:** the wettest air of all cases (15.4–17.3 g/kg), rising sharply between hours 18 and 42, with a small pressure fall.
- **Chennai_2015:** a gentle pressure fall and a steady humidity rise from 12.3 to 13.8 g/kg after hour 18.
- **Idukki_2018:** no pressure trend over 96 hours. Humidity rose to 13.8 g/kg by hour 36, then fell to about 12.5–12.8 g/kg from hour 48, so the model showed the air *drying* during the flood days themselves (15–17 August).

Chennai and Idukki also show a sudden humidity drop in the first 6 hours (from about 14.0–14.3 to 12.5–12.8 g/kg). A jump this large right after the start may be the model adjusting to the ERA5 starting data rather than real weather, so it was not interpreted.

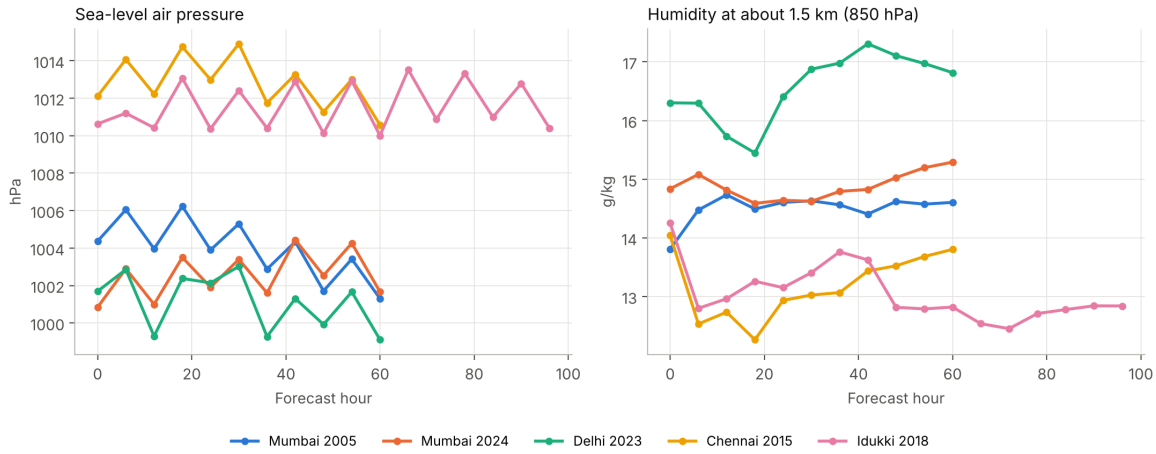


Figure 4: Pangu-Weather forecast of sea-level pressure (left) and 850 hPa humidity (right) at each city, by forecast hour.

4.8 Pangu-Weather: Warning Signs vs Flood Size

Table 6 summarises each case as the change between the first and last 24-hour averages (Section 3.6).

Table 6: Pangu-Weather warning signs (average of last 24 h minus average of first 24 h)

Case	Observed (mm)	ΔP (hPa)	Δq_{850} (g/kg)	Signal
Delhi_2023	153	−1.06	+1.11	Both present; largest moisture rise
Chennai_2015	286	−1.26	+0.72	Both present
Mumbai_2024	300	+1.17	+0.25	Weak moisture only
Idukki_2018	588	+0.35	−0.53	None (drying)
Mumbai_2005	944	−2.46	+0.17	Pressure only

Pressure change vs observed rainfall: $r = -0.447$, $p = 0.450$
Humidity change vs observed rainfall: $r = -0.615$, $p = 0.270$

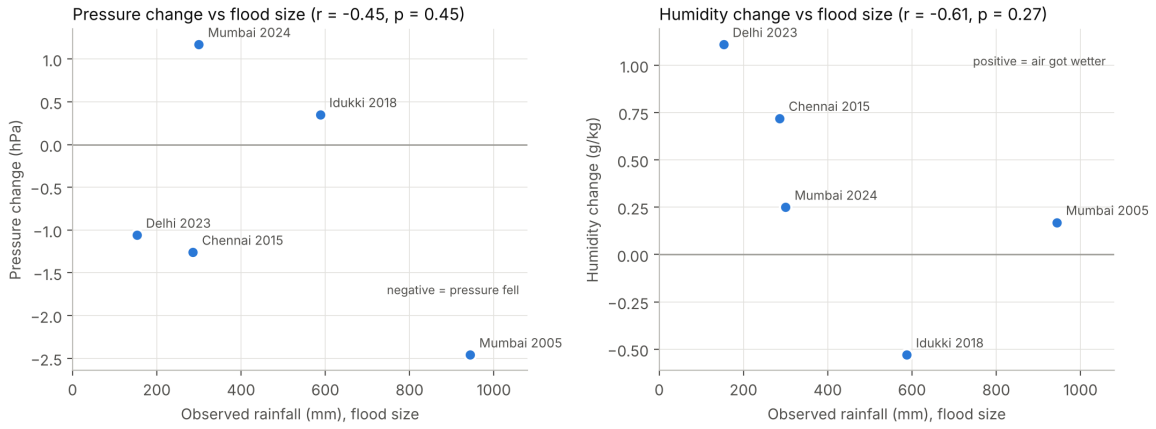


Figure 5: Pangu-Weather: pressure change (left) and humidity change (right) vs observed rainfall. A negative pressure change means the pressure fell.

Neither correlation is statistically significant. The pressure result points the expected way (bigger floods, bigger pressure falls), but this comes almost entirely from Mumbai 2005; Mumbai 2024 and Idukki both had rising pressure. The humidity result points the *wrong* way for a warning sign: the smallest flood (Delhi) had the biggest moisture rise, while the two biggest floods had little (Mumbai 2005) or negative (Idukki) change.

4.9 Computational Performance

FourCastNet ran a 72-hour (12-step) global forecast in 41 s on the T4 GPU (about 3 s per step; 51 s including model loading), not counting the time spent waiting for CDS data. Pangu-Weather, when it accidentally ran on the CPU during early testing, took about 4.3 minutes per 6-hour step, which shows how much these models depend on a GPU. Pangu-Weather's GPU speed in the CPU/GPU setup was not recorded separately, and a controlled comparison across machines (laptop CPU, Colab GPU and a lab computer) is still to be done (Section 6).

5 Discussion

Both models capture the large-scale weather. FourCastNet predicted rain increasing in the right period before each flood, and Pangu-Weather reproduced the correct regional pressure levels, a realistic twice-daily pressure cycle and a clear moisture build-up before Mumbai 2024.

FourCastNet badly underestimates heavy rain, and mainly because of its training data. FourCastNet underestimated all five floods, by 49% (Idukki) to 98% (Mumbai 2024), and the error did not depend on flood size or start date. Its rain came as steady, light amounts, never as a burst. The four-source comparison (Section 4.6) explains why: FourCastNet reproduces ERA5 rainfall fairly closely, but ERA5 itself has far less rain than IMD's measured grid at the same 25 km scale. A model trained to copy ERA5 cannot do better than ERA5. Training on average errors also pushes AI models towards smooth, moderate values instead of sharp peaks. The best result came from Idukki, a long and widespread rain event that ERA5 also captured reasonably well.

Pangu-Weather's warning signs do not scale with flood size. The strongest moisture signal came before the smallest flood, and Idukki showed no warning at all, with the air drying during the flood. Heavy rain needs moisture and also something that pushes air upward, such as mountains or converging winds. The 2018 Kerala rain was strongly driven by monsoon winds hitting the Western Ghats, which single-point pressure and humidity cannot show.

Overall. Tested with different, suitable methods, both models point to the same conclusion: they capture the large-scale state of the atmosphere, but they cannot on their own tell how intense the rain will be in a flood. For FourCastNet, most of this limit comes from the ERA5 data it learned from. Better flood forecasts from such models need training on better rainfall data, not only better model design. None of the statistical tests was significant with five events, so this is a consistent, physically sensible pattern rather than a proven result.

6 Limitations

1. **Few events.** Five events are too few for reliable correlation tests.
2. **Different kinds of observations.** The reported values are single gauges (Mumbai, Delhi), a city average (Chennai) and a basin total (Idukki). The IMD grid comparison reduces this problem but does not remove it.
3. **Idukki window.** The three IMD days used for Idukki are an assumption based on the reported 15–17 August.

4. **Mumbai 2024 window.** ERA5 is hourly, so the 19:30–01:30 UTC window is off by half an hour at each end for ERA5, and the IMD grid (daily) could not be used.

5. **Nearest grid cell.** For Mumbai, the nearest IMD cell with data was 19.00°N 73.00°E, slightly east of Santacruz, because cells nearer the coast are sea.

6. **Single-point warning signs (Pangu-Weather).** Only two variables at one grid cell were used. Variables describing lifting and moisture transport (for example wind convergence or moisture flux) and maps instead of single points would give a fuller picture.

7. **Training-period overlap.** The 2005, 2015 and 2018 events fall inside the models' training period (ERA5 up to about 2017–2018). The 2023 and 2024 events come after it and are cleaner tests; both were still heavily underestimated.

8. **Inference only.** The published trained weights were used. Retraining either model (originally about 10^3 GPU-days) is not possible with free resources.

7 Streamflow Prediction with NeuralHydrology

7.1 Objective

The second part of this work moves from forecasting the weather to predicting its effect on a river: daily river flow. A deep learning rainfall–runoff model was trained with the NeuralHydrology framework, first on a standard benchmark basin to check the pipeline, and then on an Indian catchment. The main question was whether the **NCEP FNL global analysis** (NSF NCAR GDEX dataset `d083002`) can replace catchment-specific Indian weather data as input to such a model.

7.2 Framework and Model

NeuralHydrology (Kratzert et al., 2022) is an open-source PyTorch library for deep learning in hydrology. Each experiment is fully described by a YAML configuration file (basins, periods, inputs, model and training settings), and each run saves the trained weights, the input scaling values and the evaluation results in its own folder.

All experiments used the **CudaLSTM** model with a **regression head**. A Long Short-Term Memory network (Hochreiter and Schmidhuber, 1997) reads the weather inputs for the previous 365 days and predicts the river flow on the last day. Its memory can hold information for months, which suits catchments where rain is stored in soil and groundwater before it reaches the river (Kratzert et al., 2018). Every input is scaled using the mean and standard deviation of the training period, and these values are saved with the model so test data are scaled the same way.

All runs used the same settings (Table 7), taken from NeuralHydrology's official single-basin tutorial. The small hidden size limits overfitting when only one basin is available. No tuning was done.

Table 7: Model and training settings used in all NeuralHydrology runs

Setting	Value
Model / head	CudaLSTM / regression (linear output)
Hidden size	20
Initial forget-gate bias	3
Output dropout	0.4
Input sequence length	365 days (predicting the last day)
Optimiser / loss	Adam / mean squared error
Learning rate	0.01 (epochs 1–29), 0.005 (30–39), 0.001 (40–50)
Batch size / epochs	256 / 50
Gradient clipping	norm 1
Hardware	CPU (Windows laptop and Google Colab)

7.3 Evaluation Metrics

With Q_o the observed and Q_s the predicted daily flow:

- **Nash–Sutcliffe efficiency** (Nash and Sutcliffe, 1970):

$$\mathrm{NSE} = 1 - \frac{\sum_t (Q_s^t - Q_o^t)^2}{\sum_t (Q_o^t - \overline{Q_o})^2}.$$

  NSE = 1 is perfect; NSE = 0 means the model is no better than predicting the average flow every day; negative values are worse than that.

- **Kling–Gupta efficiency** (Gupta et al., 2009):

$$\mathrm{KGE} = 1 - \sqrt{(r-1)^2 + (\alpha-1)^2 + (\beta-1)^2}, \qquad \alpha = \frac{\sigma_s}{\sigma_o}, \quad \beta = \frac{\mu_s}{\mu_o},$$

  which balances timing (correlation r), variability (α) and volume (β). KGE = 1 is perfect; KGE > -0.41 is better than predicting the average flow (Knoben et al., 2019).

- **RMSE**: typical error in mm/day. **Pearson** r: how well rises and falls are timed, regardless of size.

- NeuralHydrology's **Alpha-NSE** (σ_s/σ_o, ideal 1) and **Beta-NSE** ($(\mu_s - \mu_o)/\sigma_o$, ideal 0) describe errors in variability and in average flow.

7.4 Stage 1: Checking the Pipeline on CAMELS US

The pipeline was first checked on the CAMELS US dataset, following NeuralHydrology's introductory tutorial with one basin (USGS gauge 01022500) and daily weather data from three products (Daymet, Maurer and NLDAS). The model was trained on 1 Oct 1999–30 Sep 2008, validated on 1980–1989 and tested on 1 Oct 1989–30 Sep 1999. The tutorial places the test period *before* the training period; this was kept only to reproduce the reference result.

The test NSE was **0.776**, matching the tutorial's reference value (0.777). This confirmed that the installation, data layout and training and evaluation steps were correct.

7.5 Stage 2: CAMELS-IND with Its Own Weather Data

Dataset. CAMELS-IND provides daily catchment-averaged weather data and fixed catchment properties for 472 catchments in Peninsular India; 228 of them have observed river flow

for more than 30% of 1980–2020. Rainfall and temperature come from IMD gridded products based on gauges and stations, most other variables (radiation, wind, humidity, evaporation, soil moisture) from the IMDAA regional reanalysis (about 12 km), and river flow from CWC gauges via India-WRIS. The raw files were converted with NeuralHydrology's built-in `preprocess_camels_ind_dataset` function, which joins weather and flow data, converts flow from m^3/s to mm/day using each catchment's area, writes one file per basin and combines the property tables.

Experiment. Basin 3002 was chosen for its long record: 8,226 days of observed flow between 1 Jun 1993 and 8 Oct 2018. The model used all 18 weather variables and a time-ordered split: training 1 Jun 1993–31 Dec 2008, validation 2009–2011, test 1 Jan 2012–8 Oct 2018.

Result. Validation NSE settled at 0.60–0.63, but test NSE was only **0.0037**. The test period had 1,438 days with observed flow, more than half of them with zero flow, and a maximum of 946 mm/day. As Section 7.6.6 shows, this maximum belongs to a set of wrong gauge values, so the near-zero test NSE of this run is most likely caused by those values rather than by the model. The run was not re-evaluated with them removed.

7.6 Stage 3: NCEP FNL Data from GDEX (d083002)

7.6.1 Dataset

NCEP FNL (Final) Operational Global Analyses (NCEP, 2000) come from the Global Data Assimilation System. They are available every 6 hours on a $1° \times 1°$ global grid from July 1999, and are shared by NSF NCAR's Geoscience Data Exchange (GDEX) as dataset `d083002`.

An important finding from GDEX's own catalogue is that **rainfall (PRATE) is available in this dataset only from 12 June 2019**. The CAMELS-IND flow record ends in 2020 (for basin 3002 in October 2018), so FNL cannot provide rainfall for any usable training period. Two moisture variables that cover the whole period were used as indirect rainfall information instead: **precipitable water** (all the water vapour in the air column) and **soil moisture** (0–0.1 m layer). Table 8 lists all variables used.

Table 8: FNL variables requested from GDEX and the daily inputs made from them

FNL variable	Level	Daily model input
Temperature (TMP)	2 m above ground	`tmax`, `tmin`, `tavg`: max, min and mean of the four 6-hourly values; K → °C
Relative humidity (R H)	2 m above ground	`rel_hum`: daily mean (%)
Wind components (U GRD, V GRD)	10 m above ground	`wind`: daily mean of $\sqrt{u^2 + v^2}$ (m/s)
Precipitable water (P WAT)	Entire atmosphere	`pwat`: daily mean ($kg\,m^{-2}$ = mm)
Soil moisture (SOILW)	0–0.1 m below ground	`soilw`: daily mean (m^3/m^3)
Precipitation rate (PRATE)	Surface	*Not used: available only from 12 June 2019*

7.6.2 Data Access and Processing

7.6.3 Data Access and Processing

FNL data were obtained through the GDEX API as server-side subsets covering a $2° \times 2°$ box around the basin (approximately 19.3°N, 81.8°E), in NetCDF format. Because each request can contain only one type of vertical level, the variables were requested in four groups (2 m, 10 m, entire atmosphere and soil layer), one year at a time. For each 6-hourly time step, the

value of the grid cell nearest to the basin was extracted. The download was automated in a Google Colab notebook, which tests each request on two days of data before the full download and resumes automatically after interruptions.

The data cover 1 Jan 2011 to 8 Oct 2018 and are complete: 11,352 six-hourly time steps (2,838 days × 4), with no missing days. The 6-hourly values were aggregated to daily values as described in Table 8. Table 9 summarises the resulting daily inputs.

Table 9: Summary of FNL daily inputs, 1 Jan 2011 – 8 Oct 2018 (2,838 days)

Variable	Mean	Std. dev.	Min	Max
`tmax` (°C)	30.71	4.55	19.95	45.45
`tmin` (°C)	19.53	4.57	7.88	29.95
`tavg` (°C)	24.61	4.23	13.86	35.93
`rel_hum` (%)	60.05	20.89	12.35	88.73
`wind` (m/s)	1.98	0.71	0.42	7.39
`pwat` (mm)	33.87	17.03	4.25	64.88
`soilw` (m^3/m^3)	0.18	0.08	0.05	0.37

7.6.4 Comparison with CAMELS-IND Data

Figure 6 compares the FNL inputs with the CAMELS-IND data for the same basin. FNL precipitable water rises and falls with the monsoon in step with the observed rainfall, so FNL captures the timing of the wet season. Day by day, however, its link with CAMELS-IND rainfall is only moderate: correlation 0.42 for precipitable water, 0.41 for soil moisture and 0.42 for relative humidity. These variables show that rain is likely, not on which day it falls or how much. FNL temperatures follow the same seasonal cycle but are about 1–2 °C cooler, as expected when one 1° grid cell (about 100 km across) is compared with an average over the actual catchment.

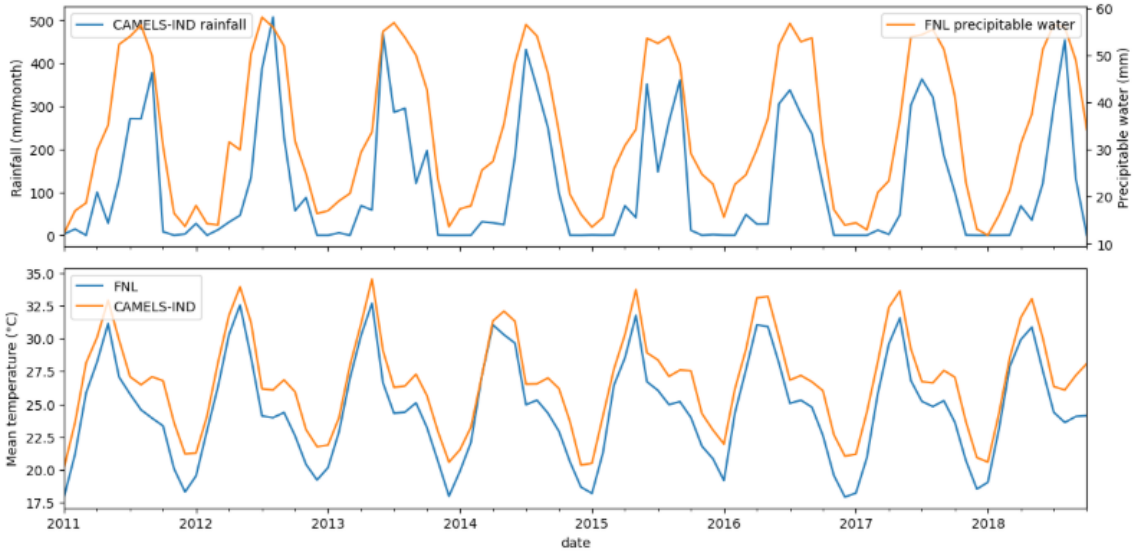


Figure 6: Monthly CAMELS-IND rainfall and FNL precipitable water (top; separate axes, left and right) and monthly mean temperature from both sources (bottom), basin 3002, 2011–2018.

7.6.5 Experimental Design

Three models were trained that differ *only* in their weather inputs (Table 10); the target (observed CAMELS-IND flow), basin, periods, model and settings were the same. This isolates

the effect of the input data. Setup B tests whether FNL becomes competitive once its missing variable, rainfall, is added from observations.

Table 10: The three experimental setups

Setup	Weather inputs
A: FNL only	FNL `tmax`, `tmin`, `tavg`, `rel_hum`, `wind`, `pwat`, `soilw` (7)
B: FNL + CAMELS-IND rainfall	As A, plus CAMELS-IND `prcp` (8)
C: CAMELS-IND only	CAMELS-IND `prcp`, `tmax`, `tmin`, `tavg`, `rel_hum`, `wind` (6)

The periods were: warm-up 2011 (the 365-day input history for the first training day), training 2012–2015, validation 2016 and test 1 Jan 2017–8 Oct 2018. A day can be used only if it has observed flow and complete inputs for the previous 365 days. Basin 3002's flow record has gaps, so this gave **889 training, 142 validation and 407 test samples** for each setup.

7.6.6 Results

Table 11 shows the test scores as first computed. All three setups scored an NSE of about zero and an RMSE above 104 mm/day, and the three were almost identical.

Table 11: Test scores (1 Jan 2017 – 8 Oct 2018), all 407 days

Setup	NSE	KGE	RMSE	Pearson r	Alpha-NSE	Beta-NSE
A: FNL only	−0.008	−0.596	104.589	0.080	0.018	−0.103
B: FNL + IND rainfall	−0.008	−0.606	104.602	0.087	0.020	−0.107
C: CAMELS-IND only	−0.010	−0.632	104.690	0.056	0.017	−0.108

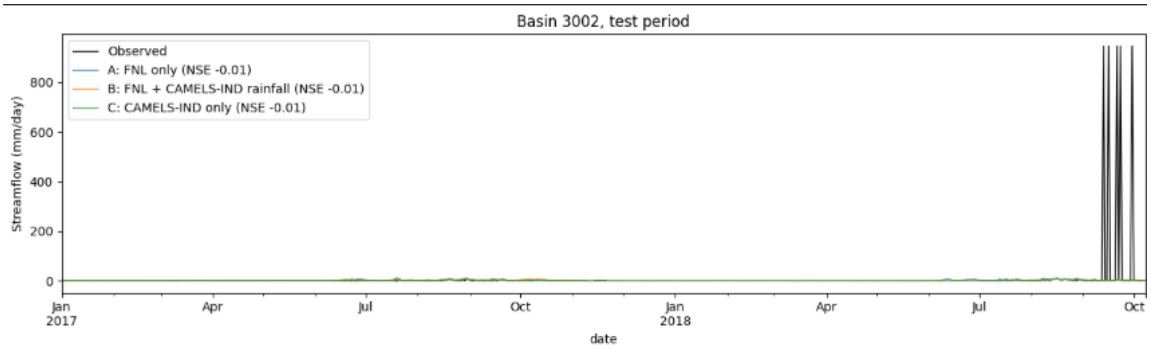


Figure 7: Observed and predicted river flow over the test period, including all observed values. Five isolated values of 946 mm/day in September 2018 dominate the plot.

Data quality finding. The hydrograph (Figure 7) explains these scores. On five days (13, 16, 21, 23 and 30 September 2018) the observed flow is *exactly* 946.46722 mm/day, while on every other test day it stays below about 11 mm/day. The same value to five decimal places on separate days, and a flow equal to almost a metre of water over the whole catchment per day, point to recording or processing errors, not real floods. These five values dominate both the squared errors and the variance of the observations, which pushes every model's NSE towards zero regardless of how well it does on other days. All values above 100 mm/day were therefore treated as errors and left out of the evaluation (only these five days were above that threshold anywhere in the record). None of them is in the training period, so the trained models are

not affected and no retraining was needed. The same values also fall in the test period of the Stage 2 run (Section 7.5).

Table 12 and Figure 8 give the scores without these values, and Figure 9 the matching hydrograph.

Table 12: Test scores with the five wrong days removed

Setup	NSE	KGE	RMSE (mm/day)	Pearson r
A: FNL only	−0.301	0.007	1.711	**0.634**
B: FNL + CAMELS-IND rainfall	−0.390	0.239	1.769	0.574
C: CAMELS-IND only	−0.193	**0.380**	**1.639**	0.531

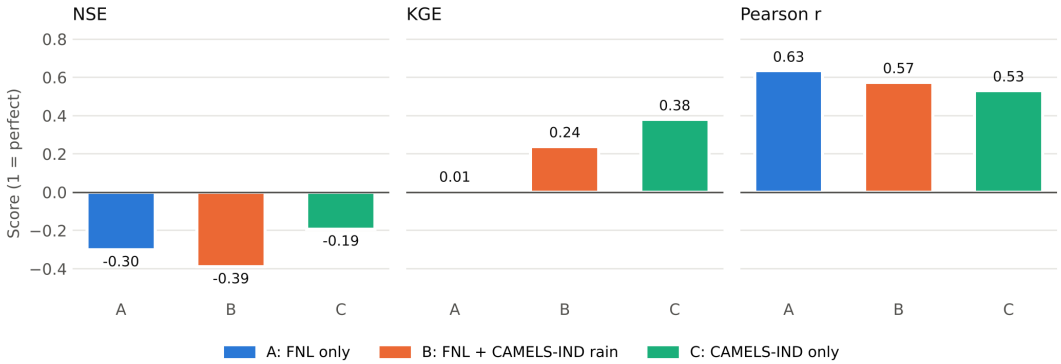


Figure 8: NSE, KGE and Pearson correlation of the three setups over the test period, with the wrong values removed (1 = perfect for all three scores).

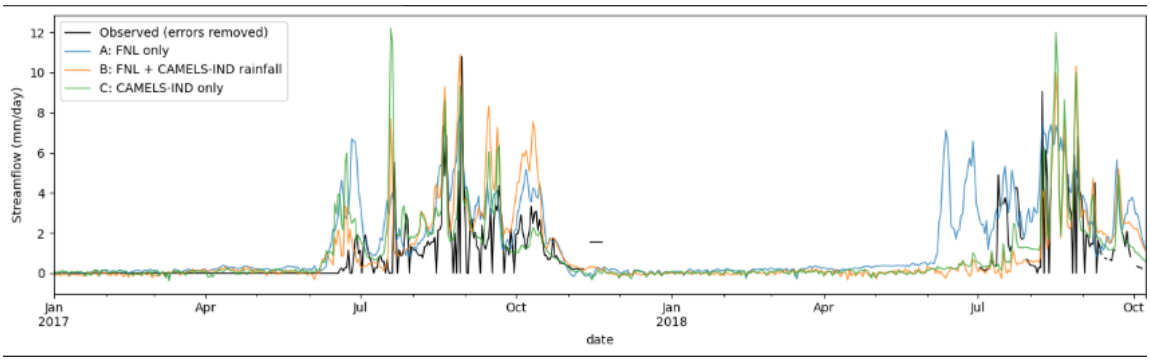


Figure 9: Observed river flow (wrong values removed) and the predictions of the three setups over the test period.

7.6.7 Discussion

CAMELS-IND data works best overall. Setup C has the highest NSE (−0.19), the highest KGE (0.38) and the lowest RMSE (1.64 mm/day). Its inputs combine gauge-based rainfall with catchment-averaged values, the information most directly linked to river flow.

FNL alone gets the timing but not the amount. Setup A has the highest correlation ($r = 0.63$): the moisture signal in FNL is enough for the model to know when the monsoon response happens. Its KGE of about zero, however, comes from a large overestimate of flow. This is clearest in June and July 2018 (Figure 9), when Setup A predicts a lot of flow while the river is still almost dry. Precipitable water and soil moisture rise as soon as the monsoon

arrives, but in reality the first rains are soaked up by dry soil and stored in reservoirs before the river responds, and the model cannot know this without rainfall amounts.

Adding rainfall to FNL did not reach the CAMELS-IND result. Setup B improved KGE over Setup A (0.24 vs 0.01) but had the lowest NSE. With only 889 training samples, eight inputs give the model more room to fit noise; part of the difference may also be random variation between single training runs.

All NSE values are negative. This means that all three models are, on average, slightly less accurate than simply predicting the average flow every day. The main reason lies in the observed data rather than in the models. As Figure 9 shows, the measured flow often drops suddenly to zero on single days during the monsoon, even when it is raining. Natural rivers do not behave this way. This suggests that the flow is controlled by dams that store and release water, or that the gauge record has gaps. A model that only sees weather data cannot predict such human-controlled changes. The developers of CAMELS-IND found the same: their LSTM, trained on 159 Indian catchments, reached a median test NSE of 0.59, but performed poorly in catchments with many dams or seasonal rivers (Mangukiya et al., 2025).

7.6.8 Limitations

1. **One basin.** All conclusions rest on one catchment whose flow record has gaps and seems regulated; they may not apply elsewhere.
2. **Each model was trained only once.** Training an LSTM involves some randomness: the model starts from random values and sees the training data in a random order. If the same model is trained twice on the same data, the scores come out slightly different. So part of the differences between Setups A, B and C may be due to chance rather than to their inputs. Training each setup several times and averaging the scores would show which differences are real.
3. **No FNL rainfall.** FNL rainfall is not available before 12 June 2019, so Setup A had to rely on indirect moisture variables.
4. **Nearest grid cell.** FNL values came from the single 1° cell nearest to the basin, not an average over the catchment.
5. **Daily values from four samples.** Daily maximum and minimum temperatures came from four 6-hourly snapshots and may miss the true extremes.
6. **Data quality handling.** The wrong values were removed using a fixed threshold of 100 mm/day; the whole flow record was not checked systematically.
7. **Short validation period and no tuning.** The validation period had only 142 samples, and the settings were taken from the tutorial without tuning.

7.6.9 Conclusion

The NeuralHydrology pipeline was checked on CAMELS US (test NSE 0.776) and extended to Indian catchment data from CAMELS-IND and to NCEP FNL data downloaded from GDEX. For basin 3002, catchment-specific CAMELS-IND data gave the best river-flow predictions (KGE 0.38), while FNL alone matched the timing of the monsoon response ($r = 0.63$) but greatly overestimated flow (KGE 0.01). The main reason is that FNL has no rainfall before June 2019, so the model has to guess rainfall from atmospheric and soil moisture. Two more findings concern the data rather than the models: the basin's flow record contains wrong values that explain the near-zero NSE of the first CAMELS-IND run, and the observed flow shows signs of regulation that no weather-driven model can reproduce. With one basin and single training runs, these results show a consistent pattern rather than a proven ranking.

8 Conclusion

Both Pangu-Weather and FourCastNet can be run as real forecasting tools with free cloud GPUs and open climate data (CDS/ERA5). FourCastNet made a three-day global forecast in under a minute on a GPU, supporting the main speed claim of these models; Pangu-Weather needed a mixed CPU/GPU setup to fit into the memory of a free T4 GPU.

For flood rainfall, only FourCastNet can be used directly, because Pangu-Weather does not forecast rain. Across five Indian floods, FourCastNet predicted rain at about the right time but far too little of it: 49% too little for the three-day Idukki event and 85–98% too little for the others, whatever the flood size or start date. Compared with IMD's gauge-based grid at the same 25 km scale, it was 82–87% too low for three of four daily events. Most of this gap is inherited from ERA5, which already had 31–85% less rain than the IMD grid and which FourCastNet reproduces fairly closely. Pangu-Weather produced realistic pressure and humidity fields, but its warning signs did not match flood severity, and the biggest floods showed the weakest moisture signals.

So current AI weather models capture the large-scale weather of Indian flood events, but should not be used alone to judge how intense the rain will be. Their accuracy is limited by the data they learn from as much as by the models themselves. This conclusion is offered with caution, given only five events. Suggested further work: more events, more warning-sign variables, models trained on better rainfall data (such as IMD grids or satellite rainfall), and a controlled speed comparison across machines.

Both parts of this work point to the same theme: global, coarse data describe the large-scale monsoon well, but accurate rainfall amounts at the local scale remain the limiting factor.

References

1. Bi, K., Xie, L., Zhang, H., Chen, X., Gu, X., & Tian, Q. (2023). Accurate medium-range global weather forecasting with 3D neural networks. *Nature*, 619, 533–538.

2. Pathak, J., et al. (2022). FourCastNet: A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators. *arXiv:2202.11214*.

3. Hersbach, H., et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999–2049.

4. Pai, D. S., Sridhar, L., Rajeevan, M., Sreejith, O. P., Satbhai, N. S., & Mukhopadhyay, B. (2014). Development of a new high spatial resolution ($0.25^\circ \times 0.25^\circ$) long period (1901–2010) daily gridded rainfall data set over India and its comparison with existing data sets over the region. *MAUSAM*, 65(1), 1–18.

5. India Meteorological Department (IMD) – station rainfall records for Santacruz (Mumbai) and Safdarjung (Delhi), and event reports for Chennai and Idukki.

6. ECMWF `ai-models` toolkit and the `ai-models-panguweather` / `ai-models-fourcastnet` plugins (open-source, GitHub).

7. `imdlib`: Python package for reading IMD gridded data (open-source, PyPI).

8. Kratzert, F., Gauch, M., Nearing, G., & Klotz, D. (2022). NeuralHydrology – A Python library for Deep Learning research in hydrology. *Journal of Open Source Software*, 7(71), 4050.

9. Kratzert, F., Klotz, D., Brenner, C., Schulz, K., & Herrnegger, M. (2018). Rainfall–runoff modelling using Long Short-Term Memory (LSTM) networks. *Hydrology and Earth System Sciences*, 22, 6005–6022.

10. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.

11. Mangukiya, N. K., Kumar, K. B., Dey, P., Sharma, S., Bejagam, V., Mujumdar, P. P., & Sharma, A. (2025). CAMELS-IND: hydrometeorological time series and catchment attributes for 228 catchments in Peninsular India. *Earth System Science Data*, 17, 461–491.

12. Newman, A. J., et al. (2015). Development of a large-sample watershed-scale hydrometeorological data set for the contiguous USA. *Hydrology and Earth System Sciences*, 19, 209–223.

13. Addor, N., Newman, A. J., Mizukami, N., & Clark, M. P. (2017). The CAMELS data set: catchment attributes and meteorology for large-sample studies. *Hydrology and Earth System Sciences*, 21, 5293–5313.

14. National Centers for Environmental Prediction (2000). NCEP FNL Operational Model Global Tropospheric Analyses, continuing from July 1999. NSF NCAR Research Data Archive, `https://doi.org/10.5065/D6M043C6`.

15. Nash, J. E., & Sutcliffe, J. V. (1970). River flow forecasting through conceptual models part I – A discussion of principles. *Journal of Hydrology*, 10(3), 282–290.

16. Gupta, H. V., Kling, H., Yilmaz, K. K., & Martinez, G. F. (2009). Decomposition of the mean squared error and NSE performance criteria. *Journal of Hydrology*, 377, 80–91.

17. Knoben, W. J. M., Freer, J. E., & Woods, R. A. (2019). Technical note: Inherent benchmark or not? Comparing Nash–Sutcliffe and Kling–Gupta efficiency scores. *Hydrology and Earth System Sciences*, 23, 4323–4331.